Báo cáo: Ba phiên bản dữ liệu mô phỏng và huấn luyện Evo-1 cho tay máy SO-101

Người thực hiện: Nguyễn Công Đạt

Ngày: 15/10/2026

Mã nguồn: thư mục `so101_evo1/`. Mọi số liệu dưới đây lấy từ các file kết quả liệt kê ở mục 9.

---

1. Tóm tắt

- **Mục tiêu.** Giúp policy Evo-1 hoàn thành tốt hơn 11 tác vụ gắp–đặt bằng cách train thêm dữ liệu mô phỏng (MuJoCo) cùng với dữ liệu thật (REAL_PLUS_SIM).
- **Ba phiên bản dữ liệu mô phỏng đã làm:**

Phiên bản	Số episode	Điểm mới
v1	220	cảnh mô phỏng ban đầu
v2	440	camera và màu sắc chỉnh cho giống ảnh thật
v3	1000	vật lý nồi đã sửa, vị trí vật đa dạng hơn, demo có thêm cách sửa sai (DART), dừng canh tâm và thả an toàn

- **Kết quả offline.** Sai số dự đoán action trên tập test thật giảm dần: 12,04° (v1) → 11,46° (v2) → 10,52° (v2 train tiếp) → **10,44° (v3)**. Đây là mức tốt nhất của dự án; mốc "đứng yên" là 11,96°.
- **Kết quả vòng kín.** Tỉ lệ hoàn thành tác vụ trong mô phỏng không vượt quá **2/55** ở mọi phiên bản.
  - Dữ liệu mô phỏng giúp robot tìm đúng vật và chuyển động êm hơn.
  - Dữ liệu chưa giúp robot đủ chính xác ở pha gắp và tiếp xúc với vật.
- **Kết luận.** Tiêu chí dừng đặt trước cho v3 (≥ 10/55 trên bố cục train) không đạt. Hướng "thêm dữ liệu mô phỏng" dừng lại; bước tiếp theo là chạy thử (dry-run) trên robot thật với checkpoint tốt nhất.

---

2. Giải thích thuật ngữ và tên model

Thuật ngữ	Ý nghĩa
Episode	Một lần thực hiện trọn một tác vụ, từ đầu đến cuối
Oracle	Bộ điều khiển viết sẵn, biết chính xác vị trí vật, dùng để sinh demo mô phỏng
Offline	Cho model xem ảnh và trạng thái đã ghi, rồi so action dự đoán với action thật. Không chạy robot
Vòng kín	Model điều khiển robot trong mô phỏng từ đầu đến cuối; đếm số lần hoàn thành tác vụ
Mốc "đứng yên"	Dự đoán robot giữ nguyên vị trí hiện tại. Model tốt phải có sai số thấp hơn mốc này
Ô train / ô test	Mặt bàn chia thành các ô xen kẽ. Dữ liệu train chỉ đặt vật ở ô train; đánh giá "vị trí test" dùng các ô chưa từng xuất hiện trong dữ liệu train
DART	Cách sinh demo dạy sửa sai: cố ý đẩy robot lệch, rồi ghi lại hành động đưa robot về đúng đường

Tên các model (checkpoint trong `checkpoints/evo1_so101_s2lite_selective_llm*`):

Tên model	Dữ liệu train
REAL_ONLY	chỉ dữ liệu thật (mốc so sánh)
rps, rps2	dữ liệu thật + mô phỏng **v1** (rps2 lấy gấp đôi số frame)
rpsv2, rpsv2c	dữ liệu thật + mô phỏng **v2** (rpsv2c train tiếp từ rpsv2, gấp đôi số frame)
rpsv3	dữ liệu thật + mô phỏng **v3** (train tiếp từ rpsv2c)

---

3. Thiết lập chung cho cả ba phiên bản

3.1. Dữ liệu và phần cứng

- **Dữ liệu thật:**
  - 11 tác vụ, 551 episode;
  - chia theo episode: train 432 / val 55 / test 55; loại 9 episode tĩnh (`reports/split_manifest.json`);
  - action là góc khớp tuyệt đối (độ) cho 5 khớp, cộng gripper thang 0–100.
- **Phần cứng:** GPU RTX 3050 Laptop 6 GB, RAM khoảng 11 GB.

3.2. Mô hình và cách train

- **Mô hình:** Evo-1, dùng bản tích hợp chính thức trong LeRobot 0.6.1.
  - Phần nhìn–hiểu (VLM) là InternVL3-1B (bản `014c0583`), mô hình ngôn ngữ cắt còn 14 layer.
  - Phần sinh action là action head flow-matching, mỗi lần dự đoán 50 bước (chunk 50).
  - Ảnh vào 448×448 từ 2 camera (top và cổ tay).
- **Điểm khởi đầu chung:** Stage 1, run 2 (`checkpoints/evo1_so101`).
  - Khởi tạo từ run 1, train 20.000 step × batch 32 trên 40.220 frame thật.
  - Val loss 0,0280; sai số offline trên test 11,95°.
- **Cấu hình train** `selective_llm`**:**
  - chỉ train layer 12–13 của mô hình ngôn ngữ, lớp norm cuối và action head;
  - tổng 151,9 triệu tham số được train, trong đó 29,8 triệu thuộc mô hình ngôn ngữ;
  - phần còn lại của VLM đóng băng.
  - Lý do: GPU 6 GB không đủ để train toàn bộ VLM.
- **Tăng tốc bằng cache:** hidden state ở đầu vào layer 10 được tính một lần và lưu sẵn (`scripts/cache_evo1_hidden.py`).
  - Lúc train chỉ cần chạy layer 10–13, nên tốc độ đạt khoảng 25–28 mẫu/s, thay vì khoảng 2 mẫu/s nếu chạy cả mô hình.
  - Đã kiểm tra: kết quả dựng lại từ cache khớp tuyệt đối với cách tính chính thức (sai số 0,0).
- **Siêu tham số chung** (`scripts/train_evo1_selective.py`):
  - batch 32 (8 mẫu × 4 lần cộng dồn gradient), learning rate 1e-5, lịch cosine, warm-up 500 step;
  - tính toán BF16, trọng số lưu fp32;
  - dữ liệu thật dùng chung `train_s10_o0`: 20.213 frame, lấy 1 trong 10 frame.
- **Mốc so sánh REAL_ONLY:** chỉ dữ liệu thật, cùng cấu hình, 10.000 step.
  - Offline 11,26°, tốt hơn "đứng yên" ở 9/11 tác vụ.
  - Vòng kín 0/55.

3.3. Cách đánh giá

Loại	Cách làm
Offline (dữ liệu thật)	55 episode test × 4 thời điểm = 220 lần dự đoán, mỗi lần 50 bước. Sai số là MAE trung bình trên 6 trục (5 khớp tính bằng độ, gripper thang 0–100). Mốc "đứng yên" = 11,96°
Offline (dữ liệu mô phỏng)	Cùng cách đo, trên 5 episode mô phỏng đã dùng để train của mỗi tác vụ. Cho biết model đã học thuộc demo đến đâu
Vòng kín, vị trí test	MuJoCo, 11 tác vụ × 5 episode = 55 lượt, seed 1000–1004, vật đặt ở ô test
Vòng kín, bố cục train	Đặt vật đúng như lúc sinh dữ liệu (dùng lại seed). Đã kiểm tra bố cục tái tạo khớp tuyệt đối (lệch 0,0)

- Mặc định, đánh giá vòng kín dùng BF16, 8 bước sampling flow-matching và không compile (RAM khoảng 5 GB). Trường hợp khác được ghi rõ ở từng chỗ.
- Mỗi cấu hình chỉ chạy một lần, 5 episode/tác vụ. Vì vậy chênh lệch 1–2 lượt thành công nằm trong mức nhiễu.

---

4. Phiên bản v1: dữ liệu mô phỏng ban đầu

4.1. Cách làm

- **Cảnh:** `configs/simulation.yaml`, dùng mô hình SO-101 chính thức. Vật là các khối hình học; camera top đặt theo ước lượng.
- **Sinh dữ liệu** bằng oracle dùng IK (`scripts/generate_sim_data.py`):
  - profile `position_visual` (ngẫu nhiên vị trí và ánh sáng), chỉ đặt vật ở ô train;
  - **220 episode** (20/tác vụ), 62.998 frame;
  - oracle thành công 220/240 lần (91,7%); chỉ lưu các lần thành công.
- **Train `rps`:** từ run 2, 10.000 step, thêm 6.400 frame mô phỏng (24% dữ liệu train). Mất 3,24 giờ; RAM 4,84 GB, VRAM 4,13 GB.
- **Biến thể `rps2`:** lấy thêm 6.286 frame từ chính 220 episode đó, nâng tỉ lệ mô phỏng lên 39%. Các điểm khác giữ nguyên.

4.2. Kết quả

Vòng kín trên cảnh v1, vị trí test, 32 bước sampling (cấu hình mặc định lúc đó).

	REAL_ONLY	rps	rps2
Val loss (val thật)	0,0262	0,0263	0,0267
Offline thật (sai số)	**11,26°**	12,04°	12,40°
Số tác vụ tốt hơn "đứng yên"	9/11	5/11	4/11
Thành công vòng kín	0/55	0/55	0/55
Lượt có tác động vào vật	2/55	25/55	24/55
Lượt làm đổ nồi	0	18	19

4.3. Nhận xét

- **Robot đi đúng tới vật nhiều hơn.** Với dữ liệu mô phỏng, 25/55 lượt chạm vật, so với 2/55 của REAL_ONLY. Đổi lại, sai số trên dữ liệu thật tăng khoảng 0,8°.
- **Thêm frame từ cùng 220 episode không giúp gì.** Nút thắt là **độ đa dạng của demo**, không phải số frame.
- **So sánh ảnh thật với ảnh mô phỏng cho thấy hai khác biệt rõ:** góc camera top lệch, và màu robot khác nhau (vàng so với xanh ngọc). Đây là cơ sở để làm v2.

---

5. Phiên bản v2: cảnh mô phỏng giống ảnh thật hơn

5.1. Cách làm

- **Cảnh** `configs/simulation_v2.yaml`**:**
  - camera top đặt lại phía sau-phải robot;
  - bàn và phông xoay −20°, vì robot thật đặt chéo so với mép bàn;
  - robot đổi sang màu xanh ngọc;
  - thêm một vật màu trắng ở mép ảnh, thay cho tay leader luôn xuất hiện trong ảnh thật.
- **Cách hiệu chỉnh:** đặt ảnh render cạnh ảnh thật để so (`scripts/calibrate_sim_camera.py`). Đây chỉ là khớp định tính, vì không có thông số nội của camera thật.
- **Kích thước khối giữ nguyên 25 mm.** Độ mở gripper lúc đang kẹp vật là 14,0–17,5 ở dữ liệu thật và 15,9–16,2 trong mô phỏng, nên khối thật không lớn hơn khối mô phỏng.
- **Sinh dữ liệu** (`scripts/generate_sim_v2.sh`):
  - **440 episode** (40/tác vụ), 123.056 frame;
  - oracle thành công 440/490 lần (89,8%);
  - mất 7,3 giờ CPU.
- **Train** `rpsv2`**:** từ run 2, 10.000 step, 12.497 frame mô phỏng (38%). Mất 3,32 giờ.
- **Train tiếp** `rpsv2c`**:**
  - khởi tạo từ rpsv2, chạy thêm 20.000 step với lịch learning rate mới;
  - lấy thêm 12.292 frame từ cùng 440 episode, nâng tỉ lệ mô phỏng lên 55%;
  - mất 6,67 giờ; RAM 4,98 GB, VRAM 4,13 GB.

5.2. Kết quả

	rpsv2	rpsv2c
Val loss (val thật)	0,0268	**0,0246**
Offline thật (sai số / số tác vụ tốt hơn "đứng yên")	11,46° / 7	**10,52° / 10**
Offline trên episode mô phỏng đã train (sai số / sai số bước đầu / cosine hướng chuyển động)	7,01° / 3,91° / 0,88	**3,92° / 2,22° / 0,96**
Vòng kín, vị trí test, cảnh v2	0/55 (14 lượt đổ nồi)	0/55 (15)
Vòng kín, đúng bố cục train, cảnh v2	0/55 (16)	**2/55** (14)
Vòng kín, bố cục train, dự đoán lại sau mỗi 10 bước thay vì 50	–	2/55 (13)

- **Chỉ đổi cảnh thì chưa đủ.** Model REAL_ONLY chạy trên cảnh v2 (đánh giá lúc đó dùng W8A8 có compile) vẫn 0/55, toàn bộ là hết giờ.
- **Train tiếp (rpsv2c) giảm mạnh sai số trên dữ liệu mô phỏng** (7,01° → 3,92°) và cải thiện cả dữ liệu thật, nhưng vòng kín gần như không đổi.
- **Dự đoán lại sau mỗi 10 bước** không giảm số lần đổ nồi. Vậy lỗi không nằm ở việc chạy liền 50 bước mà không nhìn lại.

5.3. Sửa vật lý nồi (chuẩn bị cho v3)

- **Lỗi phát hiện:**
  - code cũ chia đều khối lượng nồi cho đáy và thành, nên đáy chỉ chiếm khoảng 9%;
  - ma sát nồi với bàn thực tế là 1,0, vì MuJoCo lấy giá trị lớn hơn giữa hai vật, mà bàn đang là 1,0.
- **Sửa** (`configs/simulation_v3.yaml`, `src/simulation/env.py`):
  - 60% khối lượng nồi dồn xuống đáy (tổng khối lượng giữ nguyên);
  - ma sát nồi và vung với bàn đặt 0,4, kèm quyền ưu tiên tiếp xúc để giá trị này được dùng.
  - Đây là các giá trị điển hình cho thép trên gỗ, chưa đo trên vật thật.

- **Phép thử đẩy** (`scripts/check_pot_stability.py`, đẩy ngang ở miệng nồi):
  - trước khi sửa, 3/5 nồi bị lật ở lực 4,0–8,4 N;
  - sau khi sửa, cả 5 nồi đều trượt ở 2,2–3,7 N.
- **Oracle trên vị trí test** (5 tác vụ có nồi, 10 episode mỗi tác vụ): trước 47/50, sau 49/50, nên việc sửa không làm hỏng tác vụ.
- **rpsv2 trên cảnh đã sửa:** 1/55 thành công (task_11, không có nồi) và 11 lượt đổ nồi. Kết quả này không đủ để quy cho việc sửa vật lý.

5.4. Chẩn đoán vì sao robot vẫn thất bại

So quỹ đạo của rpsv2c với demo oracle trên cùng bố cục (`scripts/diag_policy_vs_oracle.py`, 110 lượt chạy):

- **Robot rời quỹ đạo của oracle ngay từ pha tiếp cận** ở 48/98 lượt có bị lệch. Trung bình cả lượt, robot cách quỹ đạo oracle 4,3 cm (trung vị).
- **Gripper đóng đúng độ cao nhưng lệch ngang 3,1 cm** so với vật (oracle chỉ lệch 0,06 cm). Lệch này lớn hơn cạnh khối 25 mm, nên gắp trượt.
- **Đổ nồi:** phát lại đúng chuỗi action đã ghi tái hiện được cả 27/27 lần đổ nồi, nên phân tích dưới đây đáng tin.
  - Task_05/06 (khối nằm trong nồi): đổ cả 20/20 lượt ngay khi gripper vào nồi. Gripper lệch tâm 0,9–2,9 cm và hướng hàm (`wrist_roll`) lệch khoảng 16°, khiến hai hàm và giá camera cổ tay va vào thành nồi.
  - Task_01/03: sau một lần gắp trượt, gripper rỗng vẫn hạ xuống nồi sâu hơn oracle khoảng 5 cm.

---

6. Phiên bản v3: dữ liệu nhắm vào đúng các lỗi đã chẩn đoán

6.1. Cách làm

- **Cảnh** `configs/simulation_v3.yaml`**:** hình ảnh như v2, cộng phần vật lý nồi đã sửa ở mục 5.3.
- **Vị trí vật đa dạng hơn** (profile `position_wide`):
  - mở rộng dải vị trí mỗi vật thêm ±2 cm và ±15°; xoay vật (yaw) ±45°, trước đây ±20°;
  - vẫn chỉ đặt vật ở ô train.
  - Lý do: với cách cũ, nồi thực tế chỉ xuất hiện ở góc 45–65°, hẹp hơn nhiều so với dải khai báo 25–65°.
  - Oracle trên profile mới thành công 44/55 lần.
- **Oracle mới** (`src/simulation/oracle.py`, `tasks.py`):
  - **DART (dạy sửa sai):** với xác suất 0,5 ở mỗi lần tiếp cận vật, robot bị đẩy lệch 2–4 cm và xoay ±15° trong 15–25 bước. Hành động ghi vào dữ liệu là hành động kéo robot về đúng đường, nên model học cách sửa chứ không học cách lệch.
  - **Dừng canh tâm (task_05/06):** dừng 15 bước phía trên miệng nồi sau khi tự canh tâm, rồi mới hạ xuống.
  - **Thả an toàn (task_01/03):** thả vật ở độ cao ngẫu nhiên từ miệng nồi +2 đến +3 cm, rồi rút tay lên ngay.
  - Kiểm thử: 59/59 unit test đạt, có test riêng cho DART.
- **Sinh dữ liệu** (`scripts/generate_sim_v3.sh`):
  - **1000 episode**: task_05/06 mỗi task 122 (vì đây là chỗ hay đổ nồi nhất), 9 tác vụ còn lại mỗi task 84;
  - 332.710 frame;
  - oracle thành công 1000/1168 lần (85,6%);
  - mất 16,5 giờ CPU.
- **Train** `rpsv3`**:**

  - khởi tạo từ rpsv2c, 20.000 step, 33.722 frame mô phỏng (62,5%);
  - mất khoảng 6,4 giờ (27,7 mẫu/s; bị tạm dừng và chạy tiếp 2 lần);
  - RAM 5,23 GB, VRAM 4,14 GB, đo ở đoạn chạy cuối.
- **Tiêu chí dừng đặt trước khi chạy:** trên bố cục train v3, thành công ≥ 10/55 **và** đổ nồi ít hơn rõ so với rpsv2c.

6.2. Kết quả

Hai model được đánh giá trên cùng cảnh v3 và cùng bộ seed.

	rpsv2c (đối chứng)	rpsv3
Val loss (val thật)	0,0246	**0,0240**
Offline thật (sai số / số tác vụ tốt hơn "đứng yên" / sai số bước đầu)	10,52° / 10 / 3,72°	**10,44°** / 9 / **3,33°**
Offline trên episode v3 đã train (sai số / sai số bước đầu)	–	6,15° / 1,97°
Vòng kín, đúng bố cục train v3	2/55 (9 lượt đổ nồi)	1/55 (6)
Vòng kín, vị trí test	1/55 (10)	2/55 (8)

6.3. Nhận xét

- **Tiêu chí dừng không đạt:** rpsv3 chỉ đạt 1/55 trên bố cục train v3.
- Số lượt đổ nồi giảm nhẹ (9 → 6 và 10 → 8), độ chính xác offline nhích lên. Tuy vậy, các chênh lệch vòng kín chỉ 1–3 lượt, nằm trong mức nhiễu.
- Dữ liệu sửa sai, dừng canh tâm và vị trí đa dạng hơn đã không chuyển thành thành công.

---

7. Bảng tổng hợp

Model	Dữ liệu mô phỏng	Offline thật	Thành công vòng kín, vị trí test	Đổ nồi, vị trí test
REAL_ONLY	–	11,26°	0/55 (cảnh v1)	0
rps (v1)	220 episode, cảnh v1	12,04°	0/55 (cảnh v1)	18
rpsv2 (v2)	440 episode, cảnh v2	11,46°	0/55 (cảnh v2)	14
rpsv2c (v2, train tiếp)	440 episode, gấp đôi frame	10,52°	0/55 (cảnh v2) · 1/55 (cảnh v3)	15 · 10
rpsv3 (v3)	1000 episode, cảnh v3 + DART	**10,44°**	2/55 (cảnh v3)	8

- Mỗi phiên bản được đánh giá vòng kín trên cảnh của chính nó (v1, v2, v3). So sánh giữa các phiên bản chỉ chặt chẽ khi cùng cảnh, như bảng ở mục 6.2.

---

8. Đánh giá chung

8.1. Đã đạt được

- Toàn bộ quy trình chạy được trên GPU 6 GB và có thể tạm dừng hoặc chạy tiếp ở mọi bước: sinh dữ liệu → gộp → cache → train → đánh giá.
- Độ chính xác offline trên dữ liệu thật tốt dần qua các phiên bản, đạt mức tốt nhất ở v3 (10,44°).
- Model học tốt hướng chuyển động trên dữ liệu mô phỏng (cosine 0,86–0,96).
- Robot tìm đúng vật và chuyển động êm hơn: độ dịch chuyển trung bình mỗi bước giảm từ khoảng 1,24° xuống khoảng 1,15°.
- Nguyên nhân thất bại được chỉ ra bằng số liệu:
  - lệch ngang khoảng 3 cm ở pha tiếp cận;
  - hướng hàm lệch khoảng 16° khi vào nồi;

  - hạ quá sâu khi thả.

8.2. Chưa đạt được

- **Tỉ lệ thành công vòng kín không vượt 2/55** ở cả ba phiên bản, trên cả vị trí train lẫn vị trí test.

8.3. Nguyên nhân có thể

- **Giới hạn GPU:** chỉ train được action head và 2 layer cuối của mô hình ngôn ngữ. Độ chính xác cỡ 1 cm ở pha tiếp xúc có thể vượt quá khả năng của cấu hình này.
- **Sai số cộng dồn:** các lệch nhỏ tích lại khi robot tự chạy nhiều bước liên tiếp.

8.4. Hạn chế của đánh giá

- Mỗi lần đánh giá chỉ có 5 episode/tác vụ, nên kết quả nhiễu khoảng ±1–2 lượt.
- Camera mô phỏng chỉ khớp với camera thật ở mức định tính.
- Thông số vật lý lấy theo giá trị điển hình, chưa đo trên vật thật.

---

9. Hướng tiếp theo và file nguồn

9.1. Hướng tiếp theo

1. Dừng hướng "thêm dữ liệu mô phỏng".
2. Calibrate robot thật, rồi chạy dry-run phần cứng với `checkpoints/evo1_so101_s2lite_selective_llm_rpsv3/best`, theo checklist trong `reports/real_robot_readiness.md`. Dry-run chỉ đọc cảm biến, motor tắt lực, không gửi lệnh chuyển động.
3. Chỉ chạy thử có chuyển động sau khi đã kiểm tra dừng khẩn cấp và được phê duyệt.

9.2. File nguồn chính

- **Cấu hình:**
  - `configs/simulation.yaml` (v1)
  - `configs/simulation_v2.yaml` (v2)
  - `configs/simulation_v3.yaml` (v3, profile `position_wide`)
- **Sinh dữ liệu:**
  - `scripts/generate_sim_data.py`
  - `scripts/generate_sim_v2.sh`, `scripts/generate_sim_v3.sh`, `scripts/topup_sim_v3.sh`
  - `src/simulation/oracle.py`, `tasks.py`, `env.py`, `scene.py`, `assets.py`
- **Train:**
  - `scripts/cache_evo1_hidden.py`
  - `scripts/train_evo1_selective.py`
  - `src/training/selective_llm.py`
  - `scripts/run_rpsv2c_pipeline.sh`, `scripts/run_v3_pipeline.sh`
- **Đánh giá và chẩn đoán:**
  - `scripts/evaluate_offline.py`, `scripts/evaluate_policy.py`
  - `scripts/diag_policy_vs_oracle.py`, `scripts/check_pot_stability.py`
- **Báo cáo dữ liệu:**
  - `reports/sim_dataset_report.md` (v1)
  - `reports/sim_v2_dataset_report.json`, `reports/sim_v2_report_task_XX.json`
  - `reports/sim_v3_dataset_report.json`, `reports/sim_v3_report_task_XX.json`
- **Checkpoint và nhật ký train:**

  - `checkpoints/evo1_so101_s2lite_selective_llm{,_rps,_rps2,_rpsv2,_rpsv2c,_rpsv3}/training_summary.json`
- **Kết quả:**
  - `reports/offline_evo1_*_test.json`
  - `reports/evaluation_evo1_*_position*.json`
  - `reports/diag_policy_vs_oracle.md`
  - `reports/pot_stability_v2.json`, `reports/pot_stability_v3.json`
  - `reports/oracle_v2/`, `reports/oracle_v3/`, `reports/oracle_v3_wide/`
- **Nhật ký đầy đủ:** `reports/status.md`